

Security Audit

LottoPGF (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	11
Audit Resources	11
Dependencies	11
Severity Definitions	12
Audit Findings	13
Centralisation	30
Conclusion	31
Our Methodology	32
Disclaimers	34
About Hashlock	35

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

OttoDAO partnered with Hashlock to conduct a security audit of their LooteryETHAdapter.sol, TicketSVGRenderer.sol, Lootery.sol, smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

LottoPGF, developed by OttoDAO, is a Web3-based platform enabling permissionless lottery-like gamified donations designed to fund public goods. Built on Ethereum, it leverages smart contracts to ensure transparency, security, and randomness. Players obtain tickets as incentivized donations to support causes they care about, with funds distributed via decentralized protocols. The platform is fully on-chain, removing intermediaries and offering customizable lottery-like gamified donations with verifiable randomness, providing a unique decentralized approach to raising funds for charitable purposes through blockchain technology.

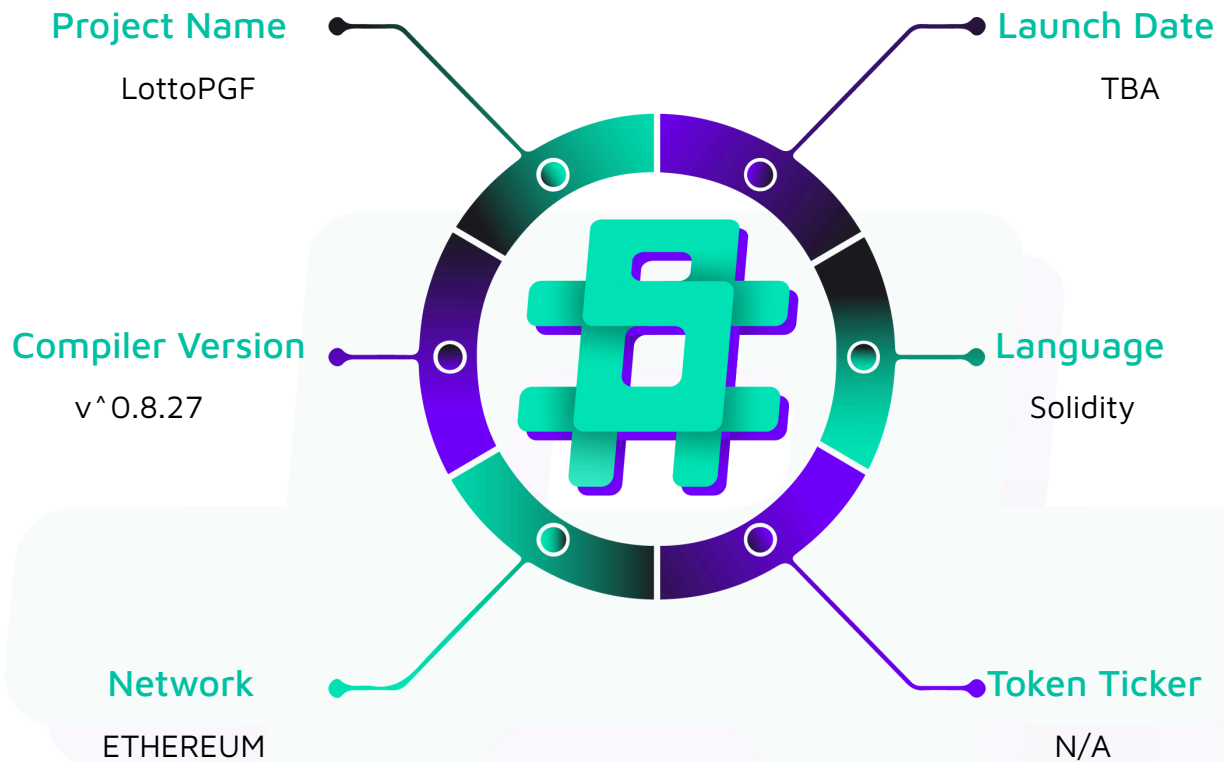
Project Name: LottoPGF

Compiler Version: v^0.8.27

Website: <https://lottopgf.org/>

Logo:



Visualised Context:

Project Visuals:

INCENTIVIZE CROWDFUNDING WITH

Permissionless Lotteries for Public Goods Funding

LottoPGF allows you to launch customisable fully onchain "Lottos" (number lotteries) to fund causes you care about, all secured on Ethereum, with verifiable randomness and unbiased winner selection

[join the waitlist](#) to fund your cause!



Lotteries Ethereum



Permissionless

Anyone can start a lottery and configure it their own way, incentivising communities to effortlessly support causes they care about.



Transparent

From randomness through winning odds, to fund distribution - everything is verifiable and auditable onchain.



Security

Our lottery factories are deployed on Ethereum L2s, meaning they are safeguarded by Ethereum's robust and secure blockchain technology.



Non-custodial

We remove financial intermediaries using smart contracts on Ethereum based blockchains. Funds are managed securely and decentralized, without relying on third parties for custody or distribution.



Truly random

We use sources of onchain randomness that are decentralised, publicly-verifiable, unpredictable & unbiased.

WE'RE ALL OTTO

ottoDAO

Capital-Formation-As-A-Service.
The missing piece on the PGF puzzle

 **hashlock.**

Hashlock Pty Ltd

Audit scope

We at Hashlock audited the solidity code within the LottoPGF project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	LottoPGF Smart Contracts
Platform	Ethereum / Solidity
Audit Date	September, 2024
Contract 1	LooteryETHAdapter.sol
Contract 1 MD5 Hash	009f39f7bf68dbae783f51c9189639e8
Contract 2	TicketSVGRenderer.sol
Contract 2 MD5 Hash	f5ef83df65fe1661b166b872c3254e84
Contract 3	Lootery.sol
Contract 3 MD5 Hash	938af76e5d90573346bf1ff76303470f
Contract 4	LooteryFactory.sol
Contract 4 MD5 Hash	f8f8e2b55e63ff400adee250b95aad7f
Contract 5	Pick.sol
Contract 5 MD5 Hash	e3eeb4402e3f980d517b946083812b3b
Contract 6	Sort.sol
Contract 6 MD5 Hash	d90a2732892218a04bb73e727a91d199

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin and solshuffle contracts. We initially identified some significant vulnerabilities that have since been addressed.



Not Secure

Vulnerable

Secure

Hashlocked

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. General security overview is presented in the [Standardised Checks](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

4 High severity vulnerabilities

2 Medium severity vulnerabilities

14 Low severity vulnerabilities

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
Lootery.sol - Allows users to: - Fund the prize - Get ticket - Draw - Redraw if some error happens - Claim the prize if won - Allows owner to: - Set ticket SVG renderer - Set gas callback limit of the Anyrand contract - Set beneficiary - Kill the game - Withdraw the fee accumulated - Rescue ETH - Rescue Token - Create ticket with any cost	Contract achieves this functionality.
LooteryFactory.sol - Allow user to: - Deploy Lootery contract - Allows admin to: - Set the implementation contract - Set the contract in charge of generating random number - Set ticket SVG renderer - Set fee recipient	Contract achieves this functionality.

LooteryETHAdapter.sol - Allow user to: - Get ticket using native token - Fund the prize using native token	Contract achieves this functionality.
TicketSVGRenderer.sol - Allow user to: - Render SVG and tokenURI	Contract achieves this functionality.

Code Quality

This Audit scope involves the smart contracts of the LottoPGF project, as outlined in the Audit Scope section. All contracts, libraries and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the LottoPGF project smart contract code in the form of GitHub access.

As mentioned above, code parts are well-commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in understanding the overall architecture of the protocol.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies

Audit Findings

High

[H-01] Lootery.ownerPick() - Should limit free tickets that the owner can get for a fairer game between player and owner

Description

According to the logic in the contract, the user can pick as many tickets as they want and the owner's pick can also be similar to the user's pick which means the user can never have a good position and has no willingness to join a game that already knows that they can never get profit

Recommendation

Consider removing ownerPick() function

Status

Resolved

[H-02] Lootery.rescueTokens() - The contract owner can rug pull the jackpot by rescuing token when the game is dead

Description

In rescueToken() function, the contract owner can withdraw all the prizeToken held by the Lootery contract if the game is dead and no ticket is sold in the last game:

```
function rescueTokens(address tokenAddress) external onlyOwner {
    uint256 amount = IERC20(tokenAddress).balanceOf(address(this));
    uint256 gameId = currentGame.id;
    if (tokenAddress == prizeToken) {
        if (
            gameId > 0 &&
            gameData[currentGame.id - 1].ticketsSold == 0 &&
```

```

        currentGame.state == GameState.Dead
    ){
        // Dead & no tickets sold -> allow rescue of everything
        // amount = amount;
        accruedCommunityFees = 0;
        unclaimedPayouts = 0;
        jackpot = 0;
    } else {
        ...
    }
}

IERC20(tokenAddress).safeTransfer(msg.sender, amount);
}

```

Vulnerability Details

Since the owner can determine when the game is dead, this `rescueTokens()` function can lead to rug-pulling by the owner. A malicious owner can wait for the game that has no ticket sold, then quickly do `kill() -> draw() -> rescueTokens()` sequence to withdraw all the prize tokens.

Recommendation

Consider changing the `rescueTokens()` function from checking if no ticket was sold in the last game to checking if no tickets were sold in the entire game:

```

function rescueTokens(address tokenAddress) external onlyOwner {
    uint256 amount = IERC20(tokenAddress).balanceOf(address(this));
    uint256 gameId = currentGame.id;
    if (tokenAddress == prizeToken) {
        if (
            gameId > 0 &&
            - gameData[currentGame.id - 1].ticketsSold == 0 &&
            > totalSupply == 0 &&
            currentGame.state == GameState.Dead
        ){
            // Dead & no tickets sold -> allow rescue of everything
            // amount = amount;
            accruedCommunityFees = 0;

```

```

        unclaimedPayouts = 0;
        jackpot = 0;
    } else {
        ...
    }
}

IERC20(tokenAddress).safeTransfer(msg.sender, amount);
}

```

Status

Resolved

[H-03] Lootery.claimWinnings() - The contract owner can rug pull the jackpot by claiming tokens when the game is dead

Description

When the game is dead and there's no winner in the last game, the unclaimed jackpot will be distributed to all the tickets sold in the last game:

```

function claimWinnings(
    uint256 tokenId
) external returns (uint256 prizeShare) {
    ...
    if (numWinners == 0 && currentGame.state == GameState.Dead) {
        prizeShare = unclaimedPayouts / (game.ticketsSold - numClaimedWinningTickets);
        unclaimedPayouts -= prizeShare;
        claimedWinningTickets[ticket.gameId].push(tokenId);
        IERC20(prizeToken).safeTransfer(whomst, prizeShare);
    }
    ...
}

```

Vulnerability Details

The jackpot is accumulated throughout games until there's a winner and the winner claims the jackpot. When the game is dead, only tickets bought at the last game can claim the jackpot, which is very unfair to other previous tickets. Let's have a scenario:

- There's a lottery contract. The current game id is #90, there was a previous winner at game #50 and claimed, so the current jackpot is accumulated in games [50,90].
- The last game before dead is 91, there's only 1 ticket bought in the game #91, and it's from user A
- After the dead, user A can claim all of the jackpot that accumulated from game [50,91], which is not fair for other ticket purchases in game [50,90]

Moreover, with a higher severity, user A can be an owner, since the owner is the one that can `kill()` the game. A malicious owner can wait for the game that has no ticket sold, and then do `kill()` -> `purchase()` -> `draw()` -> `claimWinnings()` sequence to claim all of the jackpots.

Recommendation

Consider distributing the jackpot to all the tickets that help accumulate the jackpot when the game is dead, instead of only tickets from the last round

Status

Resolved

[H-04] `Lootery._requestRandomness()` - The contract owner can make the random number not being able to draw

Description

In the `Lootery` contract, there's the `callbackGasLimit` variable, which determines how much gas is needed for the `Anyrand` contract to call `Lootery.receiveRandomWords()` function. The variable has a default value of `500_000` by `Lootery.init()` function, but can be set by `Lootery.setCallbackGasLimit()` function by the owner:

```
function setCallbackGasLimit(
```



```

    uint256 newCallbackGasLimit
  ) external onlyOwner {
    callbackGasLimit = newCallbackGasLimit;
    emit CallbackGasLimitSet(newCallbackGasLimit);
  }

```

Vulnerability Details

When requesting a random number to Anyrand contract, `callbackGasLimit` variable is used as the argument of the external call:

```

function _requestRandomness() internal {
    ...
    uint256 requestId = IAnyrand(randomiser).requestRandomness{ value: requestPrice}(block.timestamp +
30, callbackGasLimit);
    ...
}

```

Since the owner can change `callbackGasLimit` variable, a malicious owner can set it very low that the `callbackGasLimit` will not be able to cover the callback from Anyrand, which leads to `receiveRandomWords()` function can not be called. As a result, the random word can not be generated, the game state will always be at the `DrawPending` state, and the funds held by the `Lootery` contract will never be withdrawn

Recommendation

Consider removing `Lootery.setCallbackGasLimit()` function

Status

Resolved

Medium

[M-01] Lootery.forceRedraw() - Potential DOS forceRedraw() in the first random number requesting

Description

In the `Lootery.forceRedraw()` function, if the `randomnessRequest.requestId` variable is equal to 0, which intentionally means that the random number has not been requested, and then the function is reverted:

```
function forceRedraw() external payable onlyInState(GameState.DrawPending) {
    RandomnessRequest memory request = randomnessRequest;
    if (request.requestId == 0) {
        revert NoRandomnessRequestInFlight();
    }

    if (block.timestamp >= request.timestamp + 1 hours) {
        _requestRandomness();
    } else {
        revert WaitLonger(request.timestamp + 1 hours);
    }
}
```

`randomnessRequest.requestId` variable is set while requesting a random number to Anyrand contract in `_requestRandomness()` function

```
function _requestRandomness() internal {
    ...
    uint256 requestId = IAnyrand(randomiser).requestRandomness{ value: requestPrice}(block.timestamp
+ 30, callbackGasLimit);
    if (requestId > type(uint208).max) {
        revert RequestIdOverflow(requestId);
    }
    randomnessRequest = RandomnessRequest({
        requestId: uint208(requestId),
        timestamp: uint48(block.timestamp)
    })
}
```

```
});
...
}
```

Vulnerability Details

According to the Anyrand contract, the requestId is incremental and start from 0. If the first random number is requested and fails, `forceRedraw()` function will not allow it to request a new random number, because the previous `requestId` is 0. As a result, a new random word can not be generated, the game state will always be at the `DrawPending` state, and the funds held by the Lootery contract will never be withdrawn

Noted that the Anyrand contract will be deployed by the team, this issue can happen

Recommendation

Consider using `randomnessRequest.timestamp` variable to validate instead:

```
function forceRedraw() external payable onlyInState(GameState.DrawPending) {
    RandomnessRequest memory request = randomnessRequest;
    - if (request.requestId == 0) {
    + if (request.timestamp == 0) {
        revert NoRandomnessRequestInFlight();
    }

    if (block.timestamp >= request.timestamp + 1 hours) {
        _requestRandomness();
    } else {
        revert WaitLonger(request.timestamp + 1 hours);
    }
}
```

Status

Resolved

[M-02] Lootery.claimWinnings() - The claim window can be too short for the winner to claim the jackpot

Description

The game period of each game can be very short, as described in the `init()` function, it can be as short as 10 minute

```
function init(InitConfig memory initConfig) public override initializer {  
    ...  
    if (initConfig.gamePeriod < 10 minutes) {  
        revert InvalidGamePeriod(initConfig.gamePeriod);  
    }  
    gamePeriod = initConfig.gamePeriod;  
    ...  
}
```

Vulnerability Details

As per the design of the contract, a jackpot winner can only claim the prize if they just won the previous round. If the winner is not claimed, the prize will be moved to the accumulating jackpot. Although the claiming prize is permissionless, a 10-minute claiming window can be very short to notice. In an extreme case, where the chain got re-org or went down for more than 10 minutes, it will be impossible to claim the prize.

Recommendation

Consider lower bounding `gamePeriod` to a more significant amount of time

Status

Acknowledged

Low

[L-01] Lootery - Set state all values in the `gameData[currentGameId]` struct while setting only one of those value

Description

In the `_pickTickets()` function, `gameData[currentGameId]` got settled all values while there's only one value actually change. The same thing happens in the `_setupNextGame()` function.

Recommendation

```
function _pickTickets() {
    ...
-   gameData[currentGameId] = Game({
-       ticketsSold: game.ticketsSold + uint64(ticketsCount),
-       startedAt: game.startedAt,
-       winningPickId: game.winningPickId
-   });
+   gameData[currentGameId].ticketsSold = game.ticketsSold +
uint64(ticketsCount);
    ...
}

function _setupNextGame() internal {
    ...
-   gameData[gameId + 1] = Game({
-       ticketsSold: 0,
-       startedAt: uint64(block.timestamp),
-       winningPickId: 0
-   });
+   gameData[gameId + 1].startedAt = uint64(block.timestamp);
    ...
}
```

Status

Resolved

[L-02] Lootery.purchase() - Not checking if the communityFeeShare is equal to zero or not before transferring the token

Description

In the purchase() function, it is not checking if the communityFeeShare is equal to zero or not before transferring the token to the beneficiary address

Recommendation

Consider adding this condition:

```
if (communityFeeShare > 0) IERC20(prizeToken).safeTransfer(beneficiary, communityFeeShare);
```

Status

Resolved

[L-03] Lootery - Missing events in critical functions

Description

In Lootery.setTicketSVGRenderer() and Lootery.kill() functions, there's no event emitted at all. If those important state change functions don't have corresponding event emissions, it will be hard for off-chain tracking and monitoring.

Recommendation

Consider emitting events for those functions.

Status

Resolved

[L-04] Lootery.ownerPick() - Missing check if the tickets' length is not zero

Description

In the `Lootery.ownerPick()` function, it misses a check if `tickets.length == 0` just like the `purchase()` function already did.

Recommendation

Consider adding a condition to check if the ticket's length is 0 or not.

```
function ownerPick(Ticket[] calldata tickets) external onlyOwner {
    if (tickets.length == 0) {
        revert NoTicketsSpecified();
    }
    _pickTickets(tickets);
}
```

Status

Resolved

[L-05] Lootery.purchase() - Checking both `protocolFeeRecipient != address(0) && protocolFeeShare > 0` is redundant

Description

The condition if `(protocolFeeRecipient != address(0) && protocolFeeShare > 0)` is used to check if `protocolFeeRecipient` is a valid address and `protocolFeeShare` is greater than 0. But since the line above already makes sure that if `protocolFeeShare` is greater than 0 then `protocolFeeRecipient` can't be `address(0)`

```
uint256 protocolFeeShare = protocolFeeRecipient == address(0)? 0 : (totalPrice *
PROTOCOL_FEE_BPS) / 1e4;
```

Recommendation

Consider remove the condition `protocolFeeRecipient != address(0)` because it is redundant

Change the condition like below:



```
- if (protocolFeeRecipient != address(0) && protocolFeeShare > 0) {
+ if (protocolFeeShare > 0) {
```

Status

Resolved

[L-06] Lootery.setBeneficiary() - Overriding beneficiary's displayName is not emitting any event

Description

In `Lootery.setBeneficiary()` function, when overriding the display name of an existing beneficiary, it will not emit any event. It can cause the dApp to show the wrong display name in the frontend.

Recommendation

Consider emitting events when overriding the display name of an existing beneficiary

Status

Resolved

[L-07] Lootery.forceRedraw() - Function can be exposed to reentrancy

Description

The using of a low-level call in the `_requestRandomness` function below can expose to reentrancy if `msg.sender` is a malicious contract.

```
(bool success, bytes memory data) = msg.sender.call{value: excess}("");
```

By implementing a malicious contract call to `draw()` function which then calls to `_requestRandomness`. Inside the contract implement a payable function that receives the call from the contract: `msg.sender.call{value: excess}("");`

Then again the malicious call back to the `forceRedraw()` function in the lootery contract since it allows the `DrawPending` state.

Recommendation

Since this function above exposes a reentrancy. Thanks to a strictly lock state, this reentrancy can happen but still not cause a critical loss of funds. But for more safety purposes need to implement the modifier `nonReentrant()`.

Status

Resolved

[L-08] Lootery._setupNextGame() - Redundant codeblock

Description

In `Lootery._setupNextGame()` function, the logic when there's no winner and the next game is dead is exactly equivalent to when there are winners. These code blocks should be merged into one

Recommendation

Consider merging these 2 identical code blocks (in terms of logic) into one

Status

Resolved

[L-09] Lootery._setTicketSVGRenderer() - Redundant requirement

Description

In `Lootery._setTicketSVGRenderer()` function, the check if `renderer` is `address(0)` is redundant and can safely removed

Recommendation

```
function _setTicketSVGRenderer(address renderer) internal {
-   bool isValidRenderer = renderer != address(0) &&
+   bool isValidRenderer =
    IERC165(renderer).supportsInterface(
        type(ITicketSVGRenderer).interfaceId
    );
}
```

```

    if (!isValidRenderer) {
        revert InvalidTicketSVGRenderer(renderer);
    }
    ticketSVGRenderer = renderer;
}

```

Status

Resolved

[L-10] Lootery.init() - It's possible for pickLength and maxBallValue to be equal to each other

Description

In Lootery.init() function, it's possible to set pickLength and maxBallValue being equal to each other. If that happens, that means every ticket is winning, since the chance of hitting the jackpot is 100%

Recommendation

Consider making it reverted when pickLength == maxBallValue

```

function init(InitConfig memory initConfig) public override initializer {
    ...
    if (initConfig.pickLength == 0 || initConfig.pickLength > 32) {
        revert InvalidPickLength(initConfig.pickLength);
    }
    pickLength = initConfig.pickLength;

-   if (initConfig.pickLength > initConfig.maxBallValue) {
+   if (initConfig.pickLength >= initConfig.maxBallValue) {
        revert InvalidMaxBallValue(initConfig.maxBallValue);
    }
    maxBallValue = initConfig.maxBallValue;

    ...
}

```

Status

Acknowledged

[L-11] Lootery.forceRedraw() - The game state is set to its previous value which is redundant

Description

In `Lootery.forceRedraw()` function, `currentGame.state` variable is set to `GameState.DrawPending`, although its previous value is already `GameState.DrawPending`

Recommendation

Consider not set `currentGame.state` again in `Lootery.forceRedraw()` function

Status

Acknowledged

[L-12] Lootery - Redundant parameter in event

Description

In the `Lootery` contract, `RandomnessRequested` event have the `timestamp` parameter, which is emit `block.timestamp`. This is not necessary, since fetching events off-chain already has the event's timestamp

Recommendation

Consider removing `timestamp` parameter in `RandomnessRequested` event

Status

Resolved

[L-13] Lootery - Changing assert with custom error for better error tracking

Description

In the `Lootery` contract, there are various assertions using the `assert` syntax, which is not recommended for debugging

Recommendation

Consider using the custom error to replace `assert` syntax

Status

Acknowledged

[L-14] `Lootery.claimWinnings()` - Redundant NFT owner check

Description

In the `Lootery.claimWinning()` function, there's a check if the NFT owner is an `address(0)` or not. If the NFT owner is an `address(0)`, the function will be reverted. But the `_burn()` function that is later called also checks this, checking this exact check in the `Lootery.claimWinning()` function will be redundant

```
function _burn(uint256 tokenId) internal {  
    address previousOwner = _update(address(0), tokenId, address(0));  
    if (previousOwner == address(0)) {  
        revert ERC721NonexistentToken(tokenId);  
    }  
}
```

Recommendation

Consider removing NFT owner checking in the `Lootery.claimWinning()` function

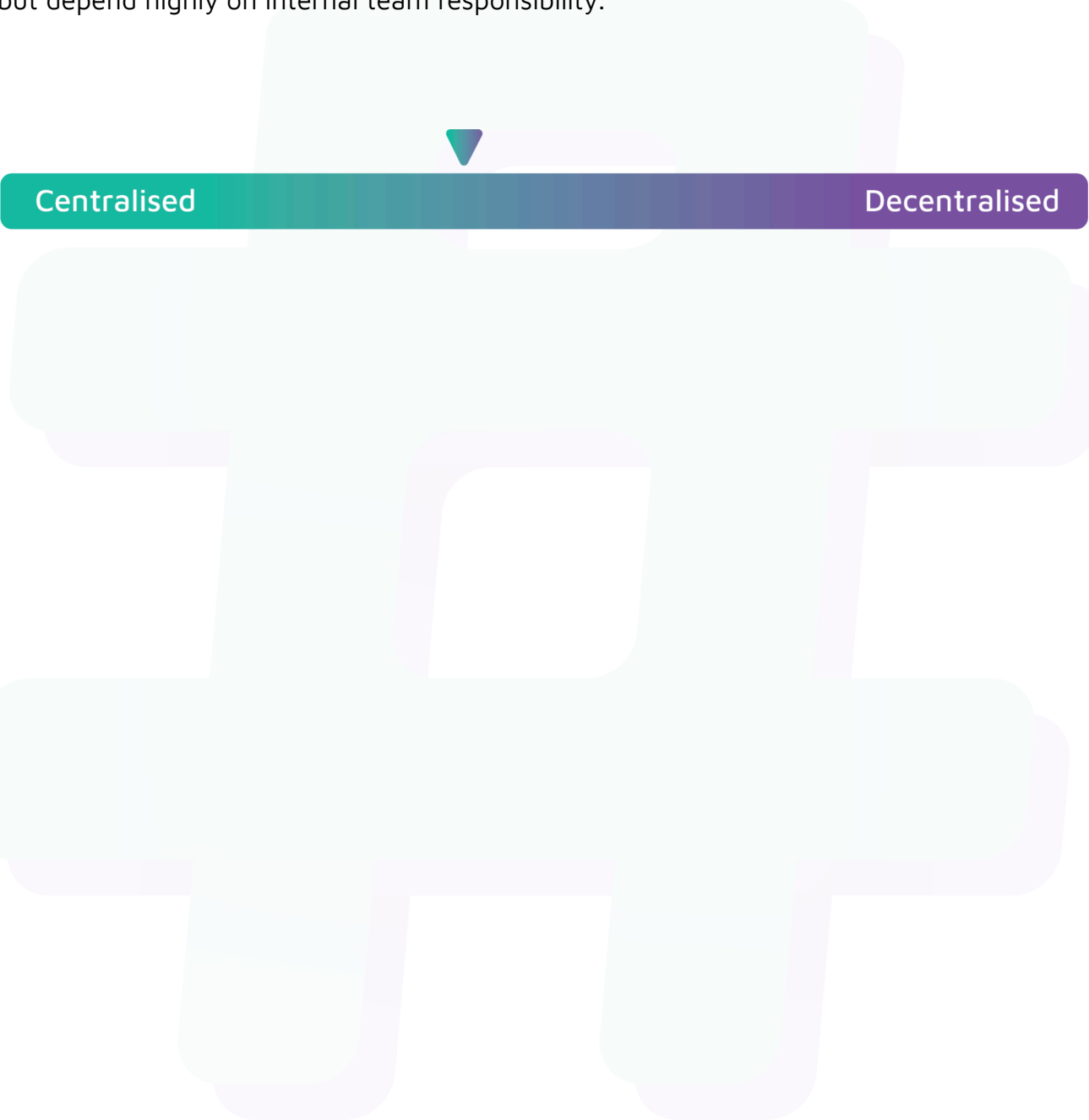
Status

Acknowledged

Centralisation

The LottoPGF project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlocks analysis, the LottoPGF project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits are to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and whitebox penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contracts details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds, and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3 oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.